

File Systems, ACLs, Permissions & Network Diagnostics

Data at rest and data in motion. A Staff-level engineer must understand how bits are physically organized on a disk, who is legally allowed to read them, and how they traverse the global network. We will deconstruct the Linux storage tree, eliminate the ambiguity of file permissions, and debug raw TCP/IP sockets.

4.0 The Virtual File System (VFS), VMs & Inodes

The VFS Abstraction & Virtual Machines (VMs)

Linux unifies radically different storage mediums under a single logical root hierarchy (`/`). Whether a directory is physically located on an NVMe SSD (Ext4), a network share (NFS), or strictly in RAM (`tmpfs`), the user interacts with it using the exact same commands. This is achieved via the **Virtual File System (VFS)** layer in the Kernel.

Architecture: VMs vs. Windows Subsystem for Linux (WSL)

Why use a Virtual Machine instead of natively installing Linux? A traditional VM (like VMware or KVM) runs a 100% isolated guest OS kernel on top of a hypervisor. Its file system is stored as a massive contiguous block on your host disk (e.g., a 50GB `.vmdk` file). It provides total security isolation, but sharing files with the host OS is painfully slow.

WSL2 takes a different approach. Microsoft runs a lightweight Linux hypervisor, but they bridge the Windows NTFS file system and the Linux VFS using the **9P (Plan 9) network protocol**. When you type `cd /mnt/c/Users` in Ubuntu, Linux is actually making a seamless 9P network request to the Windows Kernel to read the NTFS disk. This provides native Linux binary execution while retaining instant access to your Windows files.

The Inode (Index Node) Mechanic

In Linux, a filename is just an illusion for humans. The actual file is an **Inode**. The inode is a data structure on the disk that stores the file's metadata (size, owner, permissions, timestamps) and, crucially, the physical disk block pointers where the actual data resides. *The inode does not*

contain the filename. A directory is simply a special file containing a mapping of human-readable text strings (filenames) to Inode numbers.

IRL Diagnostic: The "No Space Left" Illusion

Imagine your application crashes with a `No space left on device` error. You run `df -h` and see the SSD is only 40% full. What happened?

You ran out of **Inodes**, not bytes. When you format an Ext4 partition, the OS creates a fixed number of inodes (typically 1 inode per 16KB of space). If a PHP web application creates 50 million 1-byte session files in `/var/lib/php/sessions`, you will exhaust all available inodes. The disk has 60% of its physical gigabytes free, but it mathematically cannot track any new files. You verify this using `df -i`.

4.1 Hard Links vs. Soft (Symbolic) Links

Hard Links

Because filenames are just pointers to inodes, you can have multiple filenames pointing to the exact same inode. This is a **Hard Link**. If you create a hard link, no new data is copied. The inode simply increments its "Link Count" from 1 to 2.

- If you delete the original filename, the file is *not* deleted from the disk. The Kernel just decrements the link count. The physical data is only wiped when the link count hits 0.
- **Limitations:** You cannot hard-link directories (it would create infinite recursive filesystem loops), and you cannot hard-link across different physical partitions (because Inode #1234 on `/dev/sda1` means something completely different than Inode #1234 on `/dev/sdb1`).

Soft Links (Symlinks)

A Soft Link (created with `ln -s`) is a brand new, independent Inode. Its data block does not contain file data; it literally just contains the text string path to another file (e.g., `/var/www/html/index.html`).

If you delete the target file, the soft link remains, but becomes a "dangling link" pointing to nowhere. Soft links *can* span across different hard drives and partitions.

IRL Example: Zero-Downtime Deployments

Modern deployment pipelines (like Capistrano or Laravel Envoyer) use symlinks for zero-downtime releases. The web server (NGINX) is configured to serve files from `/var/www/production/current`.

When a new version is built, it is cloned into `/var/www/production/releases/v2`. Once the build is verified, a single command is run to instantly change the `current` symlink to point to `v2`. The switch happens in a fraction of a millisecond, completely invisible to active users.

4.2 Discretionary Access Control (UGO & Special Bits)

Linux security is fundamentally based on User, Group, and Other (UGO) ownership. Every file has one owner and one group. Permissions are calculated using a strict Octal (Base-8) mathematical matrix.

The Octal Matrix

- Read (r) = 4
- Write (w) = 2
- Execute (x) = 1

By adding these numbers, we create absolute permission profiles. `6` means Read+Write (4+2). `7` means Read+Write+Execute (4+2+1). We apply three digits to represent Owner, Group, and Other.

IRL Example: Why "chmod 777" is a Fireable Offense

When junior developers face permission denied errors on a web server, they often panic and run `chmod -R 777 /var/www`. This grants Read (4) + Write (2) + Execute (1) to the Owner, the Group, and **Every single user on the system (Other)**.

If a hacker compromises a low-level service (like an FTP daemon), they can now navigate to your web root, write a malicious `backdoor.php` script, and execute it. **Correct implementation:** Directories should be `755` (rwxr-xr-x) and files should be `644` (rw-r--r--), owned strictly by the web server user (e.g., `www-data`).

The Special Execution Bits (The Danger Zone)

Standard UGO permissions have critical limitations. What if a normal user needs to change their password? The encrypted password file (`/etc/shadow`) is strictly owned by `root`. If a normal user runs the `passwd` command, the kernel should block them from saving the new password. How does it work?

- **SUID (Set-User-ID / chmod 4000):** When an executable has the SUID bit set, the process temporarily inherits the privileges of the file's *owner*, not the person executing it. Because `/usr/bin/passwd` is owned by root and has SUID set, when user "John" runs it, the process executes as `root`, allowing it to modify `/etc/shadow` safely. (Security note: If a hacker finds a

copy of `bash` with `SUID` set, they gain instant root access. Modern kernels ignore `SUID` on shell scripts for this exact reason).

- **SGID (Set-Group-ID / `chmod 2000`):** When applied to a directory, any new file created inside it automatically inherits the group ownership of the directory, rather than the primary group of the user who created it. This is mandatory for shared Git repositories or multi-developer web roots.
- **The Sticky Bit (`chmod 1000`):** Primarily used on the `/tmp/` directory. The permissions on `/tmp/` are `777` (everyone can read and write). Without the Sticky Bit, User A could maliciously delete User B's files. The Sticky Bit mandates that only the file's owner (or root) can delete or rename the file, regardless of the directory's `777` status.

4.3 POSIX Access Control Lists (ACLs)

Traditional UGO permissions are highly restrictive. What if you have a deployment directory, and you need to give User A Read/Write, User B Read-Only, and User C Execute-Only? UGO forces you to create messy, overlapping UNIX groups. **POSIX ACLs** solve this by allowing you to attach a granular list of permissions directly to a single inode.

Using setfacl and getfacl

ACLs are viewed using `getfacl filename` and modified using `setfacl`. When a file has an ACL applied, a `+` symbol appears at the end of its traditional permission string when running `ls -l` (e.g., `-rw-rwxr--+`).

IRL Implementation: The Multi-Tenant CI/CD Pipeline

Imagine a server hosting a critical financial application in `/opt/finance_app`.

1. We want the `nginx` user to only read the files to serve them.
2. We want the `jenkins_ci` user to have full write access to deploy new code.
3. We want the `audit_team` group to have read access to the logs.
4. We want to explicitly lock out the `intern` user completely.

The ACL Execution:

```
setfacl -m u:nginx:r-- /opt/finance_app
setfacl -m u:jenkins_ci:rwx /opt/finance_app
setfacl -m g:audit_team:r-- /opt/finance_app
setfacl -m u:intern:--- /opt/finance_app
```

Default ACLs: To ensure that every *new* file deployed by Jenkins automatically inherits these rules, we apply the default flag (`-d`) to the directory:

```
setfacl -d -m u:nginx:r-- /opt/finance_app
```

4.4 The Linux Networking Stack: Hardware Abstraction & iproute2

To User Space applications (like NGINX or Chrome), sending a network packet is as simple as writing bytes to a file descriptor. The Kernel's Networking Stack handles the catastrophic complexity of translating those bytes into electrical pulses on copper wire or radio frequencies in the air.

Layer 2 Abstraction: Ethernet (802.3) vs. Wi-Fi (802.11)

The kernel uses device drivers to abstract physical hardware into standardized Virtual Network Interfaces (e.g., `eth0` or `wlan0`). However, the physical physics of these mediums are radically different:

- **Ethernet (CSMA/CD):** Copper wires are a closed medium. Ethernet uses *Carrier Sense Multiple Access with Collision Detection*. The NIC listens to the wire; if it's silent, it transmits. If a voltage spike detects a collision (two machines talking at once), it aborts, waits a random microsecond interval, and retries.
- **Wi-Fi (CSMA/CA):** Radio frequencies are an open, chaotic medium. A laptop in the living room cannot physically "hear" if a smartphone in the bedroom is currently transmitting to the router. Wi-Fi uses *Collision Avoidance*. The Kernel's `mac80211` subsystem must negotiate complex RTS/CTS (Request to Send / Clear to Send) handshakes over the air before a single payload packet can be transmitted.

Because the Linux Kernel handles this at Ring 0, your Node.js application doesn't need to know if the server is plugged into a switch or connected to a 5GHz Wi-Fi network. It just writes to a TCP socket.

The Death of net-tools and the Rise of iproute2

For decades, UNIX engineers used `ifconfig`, `route`, and `arp`. These legacy tools were officially deprecated in the Linux 2.2 kernel (over 20 years ago) because they relied on the slow `ioctl` system call. Staff engineers use the **iproute2** suite, which communicates with the Kernel via ultra-fast **Netlink Sockets**.

Command Translation: net-tools vs. iproute2

- `ifconfig` → `ip addr` (View IP addresses attached to interfaces)
- `ifconfig eth0 up` → `ip link set eth0 up` (Change physical link state)
- `route -n` → `ip route` (View the Kernel's routing table)
- `arp -a` → `ip neigh` (View the ARP cache mapping IPs to physical MAC addresses)

4.5 The Name Resolution Pipeline (DNS)

When an application attempts to connect to `api.github.com`, a complex, strictly ordered resolution pipeline triggers inside the Linux OS before a single external packet leaves the machine.

The Execution Flow

1. **The Application C-Call:** The app invokes the glibc library function `getaddrinfo()`.
2. **The Name Service Switch (NSS):** The OS checks `/etc/nsswitch.conf`. It looks for the `hosts:` line, which usually reads `hosts: files dns`. This is the master rule dictating the order of operations.
3. **Local Files (The Override):** Because `files` is listed first, the OS checks `/etc/hosts`. If it finds `127.0.0.1 api.github.com` here, the pipeline terminates instantly, and traffic is routed locally. This is how engineers create local dev overrides or sinkhole malicious domains.
4. **The Resolver (systemd-resolved):** If the domain is not in the hosts file, the OS checks `/etc/resolv.conf` to find the upstream DNS server. On modern Linux, this file points to `127.0.0.53`, the local **systemd-resolved** stub resolver.
5. **External Resolution & Caching:** The stub resolver checks its local RAM cache. If it's a miss, it forwards a UDP Port 53 packet to your ISP's DNS (or 8.8.8.8), which recursively queries the Root, TLD, and Authoritative Name Servers to find the physical IP.

IRL Diagnostic: Bypassing the Cache with "dig"

If a website's IP address changes, your browser might still route to the old, dead IP because the OS resolver cached the stale record. To diagnose DNS propagation at a Staff level, you do not use `ping` (which uses the OS cache). You use `dig +trace api.github.com`.

This command bypasses your local OS cache entirely. It physically connects to the global DNS Root servers (.), follows the delegation to the `.com` TLD servers, and interrogates the final Authoritative server directly, proving exactly where the global DNS propagation is failing.

4.6 TCP State Forensics & Socket Diagnostics

TCP (Transmission Control Protocol) is a stateful, connection-oriented protocol. The Kernel tracks the exact state of every connection. Diagnosing server outages relies on understanding these states.

The "ss" (Socket Statistics) Tool

Legacy admins use `netstat`. When a server has 50,000 active connections, `netstat` halts the CPU because it has to open and parse thousands of text files inside the `/proc/net/tcp` filesystem. The modern `ss` command bypasses the filesystem entirely, using a binary **Netlink Socket** to dump the Kernel's internal TCP connection table in milliseconds.

- `ss -tlnp`: Show all (t)cp, (l)istening sockets, with (n)umeric ports, and the (p)rocess IDs that own them.
- `ss -s`: Show a high-level summary of all connection states.

The TCP Lifecycle States

Every connection transitions through strict states defined by the RFC 793 standard:

- **LISTEN**: The application is bound to a port, waiting for the 3-Way Handshake (SYN → SYN-ACK → ACK).
- **ESTABLISHED**: The handshake is complete. Data is actively flowing.

Production Outage: The TIME_WAIT Exhaustion

When a connection is gracefully closed, it does not disappear. The side that initiated the closure enters the **TIME_WAIT** state for a mandatory 60 seconds (2*MSL). The Kernel keeps the port locked to intercept any "ghost packets" that were delayed on the internet, preventing them from corrupting a new connection reusing that same port.

The Crash: A microservice makes 2,000 HTTP API calls per second to a database. It opens a port, gets data, and closes it. Because Linux only has 65,535 ephemeral ports available, the server will consume every single port in 30 seconds. The ports get stuck in **TIME_WAIT**. The server has 90% CPU and RAM free, but applications instantly crash with `Cannot assign requested address`.

The Fix: Implement Application-Level Connection Pooling (reusing a few ESTABLISHED sockets instead of creating new ones), or alter the kernel parameters (`sysctl net.ipv4.tcp_tw_reuse=1`).

Production Outage: The CLOSE_WAIT File Descriptor Leak

If `ss -s` shows thousands of sockets stuck in `CLOSE_WAIT`, it is an application-level bug.

The Mechanic: The remote client sent a `FIN` packet to say "I am done." Your OS Kernel replied with an `ACK` and put the socket into `CLOSE_WAIT`. It is now waiting for *your* application code (e.g., Python/Java) to explicitly call the `close()` function on the file descriptor.

If a developer forgot to write the `close()` method in an exception-handling block, the Kernel waits forever. Eventually, the process hits its open-file limit (`ulimit -n`) and crashes with the fatal `Too many open files` error.

4.7 The Linux Firewall: Netfilter & Packet Traversal

A firewall in Linux is not an application or a background daemon. It is a set of physical checkpoints baked directly into the Kernel's Ring 0 network stack, collectively called **Netfilter**. The `iptables` (and modern `nftables`) command-line tools are merely user-space interfaces used to insert rules into these kernel checkpoints.

The 5 Netfilter Hooks

Every packet entering, leaving, or passing through a Linux machine must traverse a strict gauntlet of hooks. If a rule at any hook dictates a `DROP`, the Kernel instantly destroys the packet in memory, saving the CPU from processing it further.

1. **PREROUTING:** The absolute first hook. The packet has hit the Network Interface Card (NIC) and entered RAM, but the OS has not yet looked at its destination IP. This is where **DNAT (Destination Network Address Translation)** occurs (e.g., port forwarding public port 80 to an internal Docker container).
2. **INPUT:** The Kernel read the routing table and realized the packet is destined for a local socket on this exact machine (e.g., your local NGINX process). This is where standard incoming security drops occur.
3. **FORWARD:** The packet is not for this machine; this machine is acting as a router. The packet is passed through to another network interface.
4. **OUTPUT:** A local user-space process (like a Python script) generated a packet to send out. The Kernel evaluates it here before routing.
5. **POSTROUTING:** The final hook before the packet hits the copper wire. This is where **SNAT (Source NAT / Masquerading)** occurs, rewriting the local sender IP to a public router IP.

The Conntrack Engine (Stateful Inspection)

Early firewalls were "stateless"—they only looked at individual packets. If you allowed port 80 outbound, you had to dangerously open all high ports inbound to allow the HTTP response back in.

Linux uses `nf_conntrack` (Connection Tracking). The Kernel hashes a 5-tuple (Source IP, Dest IP, Source Port, Dest Port, Protocol) for every single packet and stores its state in memory (visible via `cat /proc/net/nf_conntrack`). Because the Kernel knows when an outbound TCP handshake was established, you can configure the firewall to block all incoming traffic *except* packets marked with the `ESTABLISHED` state.

DNS Sinkholing via PREROUTING

Applications can be hardcoded to ignore your `/etc/resolv.conf` and query Google DNS (8.8.8.8) directly, bypassing your network security. You can forcefully intercept this at the Kernel level using the `nat` table:

```
iptables -t nat -A PREROUTING -p udp --dport 53 -j DNAT --to-destination 127.0.0.1
```

This rule intercepts any UDP packet headed to port 53 (DNS) across the entire system, regardless of its original destination, and forcefully rewrites the destination IP to `127.0.0.1`, sending it to your local Pi-hole or stub resolver. The user-space application is completely unaware the packet was hijacked.

4.8 Production Implementation: Security & Diagnostics

Implementation 1: The Bulletproof Netfilter Firewall

This Bash script implements a Staff-level, stateful firewall. It defaults to a "Default Deny" posture, prevents connection tracking exhaustion, and safely allows web and SSH traffic.

```
#!/bin/bash
set -euo pipefail

echo "[+] Flushing existing Netfilter rules..."
iptables -F
iptables -X

# 1. SET DEFAULT POLICIES (The Default Deny posture)
# If a packet doesn't match any rule below, it is ruthlessly destroyed.
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# 2. ALLOW LOOPBACK (Mandatory for local IPC)
# If you drop localhost traffic, databases and internal APIs will instantly crash.
iptables -A INPUT -i lo -j ACCEPT

# 3. STATEFUL PACKET INSPECTION (The Conntrack Miracle)
# Automatically allow incoming packets if they are part of an ALREADY established
# outbound connection (e.g., the response payload from an apt-get update).
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# 4. ALLOW CRITICAL INBOUND SERVICES
# Open SSH (Port 22). To prevent brute force, we could use the 'recent' module here.
iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j ACCEPT

# Open HTTP (80) and HTTPS (443)
iptables -A INPUT -p tcp -m multiport --dports 80,443 -m state --state NEW -j ACCEPT

# 5. DROP INVALID PACKETS (Defend against port scanning/fuzzing)
# Packets that don't conform to the TCP RFC (e.g., a SYN-ACK without a prior SYN)
iptables -A INPUT -m state --state INVALID -j DROP

echo "[+] Netfilter Kernel Hooks successfully locked down."
```

Implementation 2: The Socket Watchdog (ss + awk)

Process managers like systemd only know if the *Process ID (PID)* is alive. But what if a Java application is deadlocked? The PID is alive, but the TCP socket has stopped accepting connections. This script queries the Kernel directly to verify socket health.

```
#!/bin/bash
set -euo pipefail

TARGET_PORT=5432 # PostgreSQL

# 1. Query the Kernel's Netlink Socket via 'ss'
# -t (TCP), -l (Listening), -n (Numeric IPs)
# awk evaluates the exit code. If grep finds nothing, it triggers the failure block.
if ! ss -tln | grep -q ":{TARGET_PORT} "; then
    echo "[!] CRITICAL: Kernel reports nothing listening on Port ${TARGET_PORT}."

    # 2. False Positive Check (Is the entire network stack down?)
    # Query the Kernel routing table. If there is no default route, restarting
    # the database won't fix the issue; the network interface is down.
    if ! ip route | grep -q "^default"; then
        echo "[!] ERROR: Default gateway missing. Network interface failure."
        exit 1
    fi

    echo "[+] Network active. Initiating emergency systemd restart of database..."
    # systemctl restart postgresql
else
    echo "[+] OK: Service is bound to Port ${TARGET_PORT} and accepting connections."
fi
```

4.9 Chapter Verification, Checklist & Quiz

Staff-Level Verification Validators

- ☐ I can distinguish the physical data differences between a Hard Link (shared Inode) and a Soft Link (string pointer Inode).
- ☐ I understand how the Sticky Bit prevents unauthorized file deletion in globally writable directories like `/tmp/`.
- ☐ I can physically trace the DNS resolution order from `nsswitch.conf` through to `systemd-resolved`.
- ☐ I know how to identify Ephemeral Port exhaustion by detecting thousands of sockets in the `TIME_WAIT` state using `ss -s`.
- ☐ I understand why a developer forgetting to execute `close()` causes File Descriptor exhaustion via the `CLOSE_WAIT` TCP state.
- ☐ I can map the Netfilter traversal path, correctly placing DNAT operations in the `PREROUTING` hook and filtering in the `INPUT` hook.

Comprehensive Examination

1. Why does an SSD show 100% full when `df -h` reports 50GB of free space?

- A) The VFS layer has cached 50GB of data in RAM (tmpfs).
- B) The filesystem has exhausted its mathematical limit of Inodes due to millions of tiny files, preventing new file tracking regardless of free physical bytes.
- C) The EXT4 journal is corrupted and requires an `fsck` operation.
- D) The Kernel's connection tracking (conntrack) table is full.

2. What happens mechanically if a user executes a bash script that has the SUID (Set-User-ID) bit applied?

- A) The script executes with the privileges of the script's owner.
- B) The Kernel ignores the SUID bit entirely on interpreted scripts to prevent massive privilege escalation security holes.
- C) The script is forced to run in the background as a daemon.
- D) The script executes but requires a password prompt via `sudo`.

3. Why is the modern `ss` command physically faster at parsing 50,000 active sockets than the legacy `netstat` command?

- A) `ss` parses the `/proc/net/tcp` text file using a heavily optimized C binary.
- B) `ss` bypasses the filesystem entirely, communicating directly with the Kernel's network stack via a high-speed Netlink Socket.
- C) `ss` relies on the systemd journal for socket statistics.

4. What TCP socket state conclusively proves that a remote client has terminated the connection, but your local application code has a bug and forgot to explicitly close its end of the socket?

- A) `TIME_WAIT`
- B) `ESTABLISHED`
- C) `CLOSE_WAIT`
- D) `FIN_WAIT_2`

5. In the Netfilter framework, what is the functional difference between intercepting a packet at the `INPUT` chain versus the `PREROUTING` chain?

A) `PREROUTING` handles packets before the Kernel determines if the packet is destined for the local machine or needs forwarding. `INPUT` only handles packets mathematically destined for local user-space sockets.

B) `PREROUTING` only handles UDP; `INPUT` handles TCP.

C) `PREROUTING` is used exclusively for Source NAT (SNAT).

Systems Writing Prompts

Prompt 1: The Socket Lifecycle Debugger

An internal Node.js microservice crashes daily with a `Cannot assign requested address` error. The server has 90% of its RAM available. Detail exactly how you would use the `iproute2` suite to prove this is an Ephemeral Port / `TIME_WAIT` exhaustion issue, and explain the mechanical kernel triggers causing it.

Prompt 2: ACLs vs. Special Execution Bits

A development team requires a shared directory (`/opt/shared_code`) where User A and User B can both create and edit each other's files, but User C can only read them. Explain why traditional UGO permissions and the SGID bit fail to securely accommodate User C, and write the exact `setfacl` commands required to enforce this architecture.

CHAPTER 4 TECHNICAL ANSWER KEY

1. **(B)** Inodes act as the filesystem's index registry. Each newly allocated file consumes one Inode regardless of byte size. When the Inode table is full, the filesystem mathematically cannot map a new filename to data blocks.
2. **(B)** Historically, interpreting SUID on a bash script allowed hackers to manipulate IFS or environment variables to inject malicious commands as root. The Linux Kernel hard-codes a bypass to ignore SUID on text files starting with a shebang (`#!`).
3. **(B)** Netlink sockets provide a zero-copy, binary RPC-style communication directly between User Space and Kernel Space, bypassing the massive string manipulation overhead required to read `/proc` text files.
4. **(C)** `TIME_WAIT` indicates the local machine closed the connection gracefully. `CLOSE_WAIT` indicates the remote machine initiated the close, the Kernel acknowledged it, but the local application is failing to issue the final `close()` system call, resulting in a File Descriptor leak.
5. **(A)** The `PREROUTING` hook executes instantly upon NIC ingestion, before the Kernel parses the routing table. It is strictly used for DNAT (hijacking IPs/Ports). The `INPUT` hook only receives traffic specifically routed to a local listening socket.

Prompt 1 (TIME_WAIT Exhaustion): `ss -s` will reveal ~65,000 sockets trapped in `TIME_WAIT` . This occurs because the Kernel enforces a mandatory ~60 second cooldown on closed sockets to prevent delayed ghost packets from corrupting new connections. If an app opens and closes connections faster than this decay rate, it starves the 1024-65535 ephemeral port range.

Prompt 2 (ACL Implementation): UGO fails because assigning User C to the "Other" category or a shared group compromises granularity. Solution: Set the SGID bit to enforce group ownership (`chmod 2770 /opt/shared_code`). Then, apply ACLs: `setfacl -m u:userA:rwX,u:userB:rwX,u:userC:r-x /opt/shared_code` , and use the `-d` flag to apply these defaults to future files.